Devoir De Maison

Big Data

Rapport :

Travaux Pratiques 04 MapReduce et Analyse de Données Massives avec MongoDB

Réaliser Par :

- Aissat Mohamed Moncef.

Spécialité Data Science and Nlp

Année 2025-2026

1. Présentation des Datasets :

Pour ce projet, nous avons utilisé trois jeux de données distincts simulant des environnements de production variés :

- **pet_supplies.csv (Mission A) :** Contient des données sur des produits animaliers.
    - *Champs clés :* title, averageStar, quantity, tradeAmount, wishedCount.

- **customer_shopping_behavior.csv (Mission B) :** Un dataset "sale" (dirty data) avec des types mixtes représentant le comportement d'achat.
    - *Champs clés :* Age, Gender, Purchase Amount (USD), Review Rating, etc.

- **01_company_info.csv (Mission C) :** Informations sur des entreprises (secteur, industrie, localisation) utilisées pour enrichir les données de transactions.

2. Rôle des Fichiers du Projet :

- **import_csvs.py :** Script Python automatisant le chargement des fichiers CSV vers MongoDB. Il gère la connexion, crée la base universite et distribue les données dans les collections respectives.
- **mission_a_insights.js :** Analyse de performance commerciale (Top 10 produits).
- **mission_b_quality.js :** Script de nettoyage et de transformation des données hétérogènes.
- **mission_c_structure.js :** Analyse multidimensionnelle utilisant des jointures entre collections.
- **README.md :** Guide technique pour l'installation et l'exécution.

3. Guide d'Exécution (Commandes) :

A. Importation des données

Assurez-vous que votre serveur MongoDB est actif, puis lancez l'importation :

```
python import_csvs.py
```

B. Exécution des analyses (via mongosh)

Vous pouvez exécuter chaque mission directement depuis votre terminal :

---

mongosh --file mission_a_insights.js

mongosh --file mission_b_quality.js

mongosh --file mission_c_structure.js

---

4. Solution et Explication du code :

import_csvs.py :

Ce script Python est le moteur d'importation (ETL - Extract, Transform, Load) de votre projet. Son rôle est de prendre les fichiers bruts (CSV) et de les injecter proprement dans MongoDB pour que vos scripts JavaScript puissent travailler.

Contrairement aux autres scripts, celui-ci utilise la bibliothèque PyMongo pour communiquer avec la base de données depuis Python.

- **Code :**

```
import csv
from pathlib import Path
from pymongo import MongoClient

base_dir = Path(__file__).parent
client = MongoClient('mongodb://127.0.0.1:27017')
db = client['universite']

# Import pet_supplies.csv into sales_logs
sales_path = base_dir / 'pet_supplies.csv'
with sales_path.open(newline='', encoding='utf-8-sig') as f:
    reader = csv.DictReader(f)
    docs = []
    for row in reader:
        docs.append({
            'title': row['title'],
            'averageStar': float(row['averageStar']) if
row['averageStar'] else None,
            'quantity': int(row['quantity']) if
row['quantity'].isdigit() else None,
            'tradeAmount': row['tradeAmount'],
            'wishedCount': int(row['wishedCount']) if
row['wishedCount'].isdigit() else None,
        })
    db.sales_logs.drop()
    if docs:
```

```python
        db.sales_logs.insert_many(docs)
    print(f'Imported {len(docs)} documents into sales_logs')

# Import customer_shopping_behavior.csv into dirty_data and transactions
behavior_path = base_dir / 'customer_shopping_behavior.csv'
with behavior_path.open(newline='', encoding='utf-8-sig') as f:
    reader = csv.DictReader(f)
    dirty_docs = []
    for row in reader:
        def to_int(value):
            try:
                return int(value)
            except Exception:
                return None
        def to_float(value):
            try:
                return float(value)
            except Exception:
                return None
        dirty_docs.append({
            'customer_id': to_int(row.get('Customer ID', '').strip()),
            'age': to_int(row.get('Age', '').strip()),
            'gender': row.get('Gender', '').strip(),
            'item_purchased': row.get('Item Purchased', '').strip(),
            'category': row.get('Category', '').strip(),
            'purchase_amount_usd': to_float(row.get('Purchase Amount (USD)', '').strip()),
            'location': row.get('Location', '').strip(),
            'size': row.get('Size', '').strip(),
            'color': row.get('Color', '').strip(),
            'season': row.get('Season', '').strip(),
            'review_rating': to_float(row.get('Review Rating', '').strip()),
            'subscription_status': row.get('Subscription Status', '').strip(),
            'shipping_type': row.get('Shipping Type', '').strip(),
            'discount_applied': row.get('Discount Applied', '').strip(),
            'previous_purchases': to_int(row.get('Previous Purchases', '').strip()),
            'payment_method': row.get('Payment Method', '').strip(),
            'frequency_of_purchases': row.get('Frequency of Purchases', '').strip(),
        })
    db.dirty_data.drop()
    db.transactions.drop()
    if dirty_docs:
        db.dirty_data.insert_many(dirty_docs)
        db.transactions.insert_many(dirty_docs)
```

```
    print(f'Imported {len(dirty_docs)} documents into dirty_data and transactions')

# Import 01_company_info.csv into companies
companies_path = base_dir / '01_company_info.csv'
with companies_path.open(newline='', encoding='utf-8-sig') as f:
    reader = csv.DictReader(f)
    company_docs = []
    for row in reader:
        company_docs.append({
            'symbol': row.get('symbol', '').strip(),
            'city': row.get('city', '').strip(),
            'country': row.get('country', '').strip(),
            'industry': row.get('industry', '').strip(),
            'sector': row.get('sector', '').strip(),
            'industryKey': row.get('industryKey', '').strip(),
            'sectorKey': row.get('sectorKey', '').strip(),
            'marketCap': float(row['marketCap']) if row.get('marketCap') else None,
            'currentPrice': float(row['currentPrice']) if row.get('currentPrice') else None,
        })
    db.companies.drop()
    if company_docs:
        db.companies.insert_many(company_docs)
    print(f'Imported {len(company_docs)} documents into companies')

```

- **Explication :**

  1. Configuration et Connexion :

```
CLIENT = MongoClient('mongodb://127.0.0.1:27017')

DB = CLIENT['UNIVERSITE']
```

- **MongoClient** : Établit le pont entre Python et votre serveur MongoDB local.
- **db = client['universite']** : Sélectionne la base de données. Si elle n'existe pas, MongoDB la créera automatiquement dès l'insertion du premier document.

  2. Structure d'importation (Le pattern) :

  Pour chaque fichier CSV, le script suit le même schéma logique :

1. **Ouverture du fichier** : with path.open(...) as f.
2. **Lecture** : csv.DictReader(f) transforme chaque ligne du CSV en un dictionnaire Python (clé: valeur).
3. **Nettoyage immédiat** : Le script ne se contente pas de copier/coller. Il vérifie si les données sont valides :
   - isdigit() : Vérifie si c'est bien un nombre avant de convertir.
   - strip() : Enlève les espaces inutiles au début et à la fin des textes.
4. **Nettoyage de la base** : db.collection.drop() supprime l'ancienne collection pour repartir sur une base propre à chaque exécution.
5. **Insertion massive** : insert_many(docs) envoie tous les documents d'un coup, ce qui est beaucoup plus rapide que de les envoyer un par un.

3. <u>Focus sur les fonctions de sécurité (Mission B) :</u>

Dans la partie concernant customer_shopping_behavior.csv, le script définit deux fonctions internes très intelligentes pour gérer les données "sales" :

```
DEF TO_INT(VALUE):
  TRY:
    RETURN INT(VALUE)
  EXCEPT EXCEPTION:
    RETURN NONE
```

- **Pourquoi ?** Si le CSV contient une erreur (ex: "Vingt" au lieu de "20"), la fonction int() normale ferait planter tout le script.
- **Solution** : Le bloc try/except intercepte l'erreur et renvoie None (qui devient un null dans MongoDB) au lieu d'arrêter le programme. C'est la première étape du nettoyage demandée dans le TP.

4. <u>Rôle des Collections créées :</u>

Le script alimente quatre collections cibles :

1. **sales_logs** : Reçoit les données de pet_supplies.csv (utilisée dans la **Mission A**).
2. **dirty_data** : Reçoit les données brutes de shopping pour être nettoyées plus tard (utilisée dans la **Mission B**).
3. **transactions** : Une copie des données de shopping utilisée comme base de travail pour les jointures (utilisée dans la **Mission C**).
4. **companies** : Reçoit les infos de 01_company_info.csv (le référentiel pour les jointures de la **Mission C**).

<u>Mission A : Pôle "Insights & Tendances"</u>

- **<u>Objectif :</u>** Identifier les 10 produits les plus vendus.
- **<u>Code :</u>**

```
// Mission A - Pôle "Insights & Tendances"
// Objectif : Identifier les tendances dominantes (Top 10 produits) à partir du dataset pet_supplies.

const db = connect("127.0.0.1:27017/universite");
const coll = db.sales_logs;

const pipeline = [
  {
    $addFields: {
      quantity: { $convert: { input: "$quantity", to: "int", onError: 0, onNull: 0 } },
      averageStar: { $convert: { input: "$averageStar", to: "double", onError: 0, onNull: 0 } },
      wishedCount: { $convert: { input: "$wishedCount", to: "int", onError: 0, onNull: 0 } }
    }
  },
  {
    $group: {
```

```
        _id: "$title",
        totalQuantity: { $sum: "$quantity" },
        averageRating: { $avg: "$averageStar" },
        totalWished: { $sum: "$wishedCount" },
        count: { $sum: 1 }
      }
    },
    { $sort: { totalQuantity: -1 } },
    { $limit: 10 }
  ];

  const result = coll.aggregate(pipeline, { allowDiskUse: true
  }).toArray();
  print('--- Mission A : Top 10 produits par quantité vendue ---');
  printjson(result);

  const ratingPipeline = [
    {
      $addFields: {
        quantity: { $convert: { input: "$quantity", to: "int", onError: 0,
  onNull: 0 } },
        averageStar: { $convert: { input: "$averageStar", to: "double",
  onError: 0, onNull: 0 } }
      }
    },
    {
      $group: {
        _id: "$title",
        averageRating: { $avg: "$averageStar" },
        totalQuantity: { $sum: "$quantity" }
      }
    },
    { $sort: { averageRating: -1, totalQuantity: -1 } },
    { $limit: 10 }
  ];

  const ratingResult = coll.aggregate(ratingPipeline, { allowDiskUse: true
  }).toArray();
  print('--- Mission A : Top 10 produits par note moyenne ---');
  printjson(ratingResult);

```

Explication :

1. Connexion et Initialisation

CONST DB = CONNECT("127.0.0.1:27017/UNIVERSITE");

CONST COLL = DB.SALES_LOGS;

- **connect(...)** : Établit la connexion avec l'instance locale de MongoDB sur le port par défaut et sélectionne la base de données nommée universite.
- **db.sales_logs** : Définit la collection cible qui contient les données brutes importées du fichier CSV pet_supplies.

2. <u>Premier Pipeline : Analyse des Volumes de Ventes</u>

Ce pipeline cherche à identifier les produits les plus populaires en termes de quantité.

Étape A : $addFields (Nettoyage et Conversion)

```
$addFields: {
 quantity: { $convert: { input: "$quantity", to: "int", onError: 0, onNull: 0 } },
 averageStar: { $convert: { input: "$averageStar", to: "double", onError: 0, onNull: 0 } },
 wishedCount: { $convert: { input: "$wishedCount", to: "int", onError: 0, onNull: 0 } }
}
```

- **Problématique** : Lors d'un import CSV, MongoDB stocke souvent les nombres comme des chaînes de caractères (ex: "10" au lieu de 10). On ne peut pas faire de calculs mathématiques sur du texte.
- **Action** : On utilise $convert pour transformer les types.
    - onError: 0 : Si le champ contient du texte invalide (ex: "N/A"), il est remplacé par 0.
    - onNull: 0 : Si le champ est vide, il devient 0.

Étape B : $group (Agrégation)

```
$group: {
 _id: "$title",
 totalQuantity: { $sum: "$quantity" },
 averageRating: { $avg: "$averageStar" },
 totalWished: { $sum: "$wishedCount" },
 count: { $sum: 1 }
}
```

- **_id: "$title"** : On regroupe tous les documents qui ont le même titre de produit.
- **$sum et $avg** : Pour chaque groupe (chaque produit), on calcule la somme totale vendue, la note moyenne, et le nombre total d'ajouts aux listes de souhaits.

Étape C : $sort et $limit (Classement)

```
{ $sort: { totalQuantity: -1 } },
{ $limit: 10 }
```

- **$sort: -1** : Trie les résultats par quantité décroissante (du plus vendu au moins vendu).

- **$limit: 10** : On ne conserve que les 10 premiers résultats.

3. <u>Deuxième Pipeline : Analyse de la Qualité (Notes)</u>

L'objectif ici est de trouver les produits les mieux notés par les clients.

```
{ $SORT: { AVERAGERATING: -1, TOTALQUANTITY: -1 } }
```

- **Logique de tri** : On trie d'abord par la note la plus haute (averageRating: -1).
- **Départage** : Si deux produits ont la même note, on utilise la quantité vendue (totalQuantity: -1) comme critère secondaire pour mettre en avant le produit le plus populaire.

4. <u>Exécution et Sortie</u>

```
CONST RESULT = COLL.AGGREGATE(PIPELINE, { ALLOWDISKUSE: TRUE }).TOARRAY();

PRINTJSON(RESULT);
```

- **aggregate(pipeline)** : Lance l'exécution des étapes définies.
- **allowDiskUse: true** : C'est une sécurité. Si le volume de données dépasse 100 Mo en mémoire vive, MongoDB utilisera le disque dur pour terminer le calcul au lieu de s'arrêter en erreur.
- **toArray()** : Convertit le curseur de résultat en un tableau JavaScript lisible.
- **printjson()** : Affiche le résultat final formaté proprement dans la console.

- **<u>Le Défi :</u>** Comment assurer la précision du comptage malgré la présence de données redondantes ?
  - **La réponse:** utilisez l'étape $group sur le champ _id: "$title".
  - **Explication :** Dans les résultats, on voit par exemple pour les bols en inox (Double Pet Bowls) un count: 3. Cela signifie que le dataset contenait 3 entrées différentes pour ce même produit. En groupant par titre et en utilisant $sum: "$quantity", on a fusionné ces doublons pour obtenir un total réel de **682 882** unités. Sans ce regroupement, les statistiques seraient restées fragmentées et fausses.

<u>Mission B : Pôle "Qualité & Transformation"</u>

- **<u>Objectif :</u>** Nettoyer les données hétérogènes (erreurs de saisie, types mixtes).
- **<u>Code :</u>**

```
// Mission B - Pôle "Qualité & Transformation"
// Objectif : Nettoyer les données hétérogènes et sauvegarder le résultat
propre.

const db = connect("127.0.0.1:27017/universite");
const coll = db.dirty_data;

const pipeline = [
```

```
  {
    $addFields: {
      purchase_amount_usd: {
        $convert: { input: "$purchase_amount_usd", to: "double", onError:
null, onNull: null }
      },
      review_rating: {
        $convert: { input: "$review_rating", to: "double", onError: null,
onNull: null }
      },
      age: {
        $convert: { input: "$age", to: "int", onError: null, onNull: null }
      },
      previous_purchases: {
        $convert: { input: "$previous_purchases", to: "int", onError: null,
onNull: null }
      }
    }
  },
  {
    $match: {
      purchase_amount_usd: { $gt: 0 },
      category: { $exists: true, $ne: "" },
      item_purchased: { $exists: true, $ne: "" }
    }
  },
  {
    $group: {
      _id: null,
      averagePurchase: { $avg: "$purchase_amount_usd" },
      totalRevenue: { $sum: "$purchase_amount_usd" },
      salesCount: { $sum: 1 },
      averageRating: { $avg: "$review_rating" },
      averageAge: { $avg: "$age" }
    }
  },
  {
    $project: {
      _id: 0,
      averagePurchase: 1,
      totalRevenue: 1,
      salesCount: 1,
      averageRating: 1,
      averageAge: 1
    }
  }
];

const stats = coll.aggregate(pipeline, { allowDiskUse: true }).toArray();
```

```
print('--- Mission B : Statistiques des données nettoyées ---');
printjson(stats);

const cleanupPipeline = [
  {
    $addFields: {
      purchase_amount_usd: {
        $convert: { input: "$purchase_amount_usd", to: "double", onError:
null, onNull: null }
      },
      review_rating: {
        $convert: { input: "$review_rating", to: "double", onError: null,
onNull: null }
      },
      age: {
        $convert: { input: "$age", to: "int", onError: null, onNull: null }
      },
      previous_purchases: {
        $convert: { input: "$previous_purchases", to: "int", onError: null,
onNull: null }
      }
    }
  },
  {
    $match: {
      purchase_amount_usd: { $gt: 0 },
      category: { $exists: true, $ne: "" },
      item_purchased: { $exists: true, $ne: "" }
    }
  },
  {
    $project: {
      customer_id: 1,
      age: 1,
      gender: 1,
      item_purchased: 1,
      category: 1,
      purchase_amount_usd: 1,
      location: 1,
      size: 1,
      color: 1,
      season: 1,
      review_rating: 1,
      subscription_status: 1,
      shipping_type: 1,
      discount_applied: 1,
      previous_purchases: 1,
      payment_method: 1,
      frequency_of_purchases: 1
```

```
    }
  },
  {
    $out: "cleaned_data"
  }
];

print('--- Mission B : Exécution du pipeline de nettoyage et sauvegarde dans cleaned_data ---');
coll.aggregate(cleanupPipeline, { allowDiskUse: true }).toArray();
print('Collection cleaned_data créée / mise à jour.');
```

- **Explication :**

1. Le Premier Pipeline : Statistiques de Synthèse

Ce bloc sert à donner une vue d'ensemble de la santé financière et démographique du dataset.

Étape 1 : $addFields (La Normalisation)

C'est ici que l'on gère le côté "hétérogène" :

- **$convert** : On force les champs purchase_amount_usd, review_rating, age et previous_purchases vers des types numériques (double ou int).
- **onError: null** : Si une donnée est totalement illisible (ex: un nom écrit dans le champ âge), MongoDB mettra null au lieu de faire planter le script.

Étape 2 : $match (Le Filtrage de Qualité)

On élimine les "mauvais" documents :

- On ne garde que les achats strictement supérieurs à 0 ($gt: 0).
- On vérifie que la catégorie et l'article acheté existent et ne sont pas des chaînes vides ($ne: "").

Étape 3 : $group (Le Calcul)

- **_id: null** : On ne fait pas de groupes par produit, on veut un calcul sur **toute** la collection.
- On calcule la moyenne des prix ($avg), le revenu total ($sum), le nombre total de ventes ($sum: 1), etc.

Étape 4 : $project (Le Formatage)

- On choisit quels champs afficher dans le résultat final. _id: 0 permet de cacher l'identifiant technique pour n'avoir que les statistiques pures.

2. Le Deuxième Pipeline (cleanupPipeline) : Sauvegarde

Ce bloc est presque identique au premier, mais au lieu de calculer des moyennes, il prépare les documents individuels pour les stocker.

Étape : $project (Sélection des colonnes)

Ici, on liste tous les champs que l'on veut conserver dans notre version "propre" de la base de données. C'est comme choisir les colonnes d'un tableau Excel final.

Étape Cruciale : $out: "cleaned_data"

C'est la fonction la plus importante de cette mission :

- **Action** : Elle prend tous les documents qui ont survécu au nettoyage et les écrit dans une collection permanente nommée cleaned_data.
- **Conséquence** : Si la collection cleaned_data existait déjà, elle est écrasée par la nouvelle version. C'est idéal pour avoir une base de données "miroir" parfaitement propre prête pour être utilisée par d'autres services (ou pour la Mission C).

Le Défi : Comment gérer les documents dont les champs sont manquants sans faire planter le pipeline ?

- **La réponse:** on a utilisez l'opérateur $convert avec les paramètres onError et onNull.
- **Explication :** Dans mission_b_quality.js, on a configuré : onNull: null ou onNull: 0. Cela permet au pipeline de continuer à tourner même si une ligne n'a pas d'âge ou de prix. Au lieu de s'arrêter sur une erreur de type, MongoDB attribue une valeur par défaut ou ignore simplement la valeur dans les calculs de moyenne ($avg), ce qui sécurise le processus.

Mission C : Pôle "Structure & Corrélation"

- **Objectif :** Croiser les achats avec les données géographiques des entreprises.
- **Code :**

```
// Mission C - Pôle "Structure & Corrélation"
// Objectif : Croiser deux collections pour obtenir une analyse
multidimensionnelle.

const db = connect("127.0.0.1:27017/universite");
const orders = db.transactions;
const companies = db.companies;

const pipeline = [
  {
    $match: {
      location: { $exists: true, $ne: "" },
      purchase_amount_usd: { $gt: 0 }
    }
  },
  {
    $lookup: {
      from: "companies",
      localField: "location",
      foreignField: "city",
      as: "company_info"
    }
  },
```

```
  { $unwind: "$company_info" },
  {
    $group: {
      _id: {
        industry: "$company_info.industry",
        category: "$category"
      },
      totalRevenue: { $sum: "$purchase_amount_usd" },
      orderCount: { $sum: 1 },
      avgOrderValue: { $avg: "$purchase_amount_usd" }
    }
  },
  { $sort: { totalRevenue: -1 } },
  { $limit: 20 }
];

const result = orders.aggregate(pipeline, { allowDiskUse: true }).toArray();
print('--- Mission C : Analyse croisée industry x catégorie ---');
printjson(result);

const noLookupPipeline = [
  {
    $match: {
      purchase_amount_usd: { $gt: 0 }
    }
  },
  {
    $group: {
      _id: "$category",
      totalRevenue: { $sum: "$purchase_amount_usd" },
      orderCount: { $sum: 1 }
    }
  },
  { $sort: { totalRevenue: -1 } },
  { $limit: 20 }
];

const noLookupResult = orders.aggregate(noLookupPipeline, { allowDiskUse: true
}).toArray();
print('--- Mission C : Analyse sans jointure (comparaison de performance) ---
');
printjson(noLookupResult);
```

Explication :

1. Le Pipeline Principal (Analyse Croisée)

C'est ici que l'on mélange les données des ventes (transactions) avec les détails des entreprises (companies).

Étape 1 : $match (Filtrage initial)

On commence par nettoyer les données en entrée. On ne veut que les transactions qui ont une ville (location) et un prix valide. C'est essentiel pour que la jointure qui suit ait des données sur lesquelles s'appuyer.

Étape 2 : $lookup (La Jointure)

C'est le cœur du script.

- **from: "companies"** : On va chercher des informations dans la collection "companies".
- **localField: "location"** : On prend le nom de la ville dans notre vente.
- **foreignField: "city"** : On cherche la ligne correspondante dans la table des entreprises où la ville est la même.
- **as: "company_info"** : On range le résultat dans un nouveau champ (un tableau).

Étape 3 : $unwind (Aplatissement)

Le $lookup crée toujours un tableau, même s'il n'y a qu'une seule entreprise trouvée. $unwind "casse" ce tableau pour que l'on puisse accéder directement aux champs (comme industry) comme s'ils faisaient partie du document principal.

Étape 4 : $group (Double dimension)

Ici, on ne groupe pas par un seul champ, mais par un **objet** :

- **_id: { industry: ..., category: ... }** : Cela permet de voir, par exemple, le total des ventes pour l'industrie "Santé" uniquement sur la catégorie "Vêtements".
- On calcule ensuite le revenu total, le nombre de commandes et la valeur moyenne des commandes.

2. <u>Le Pipeline de Comparaison (noLookupPipeline)</u>

Ce deuxième bloc de code fait la même analyse mais **sans la jointure**. Il groupe uniquement par catégorie de produit.

Pourquoi faire cela ? C'est pour répondre à la partie "Rapport de Performance" de votre TP. En exécutant les deux, vous pouvez comparer :

1. **Avec jointure :** Plus riche en informations, mais plus lent (MongoDB doit chercher dans deux fichiers différents).
2. **Sans jointure :** Très rapide, mais on perd l'information sur le secteur industriel.

- **<u>Le Défi :</u>** Évaluer l'impact de la jointure sur le temps d'exécution par rapport à une requête simple.

- **La réponse dans vos résultats :** C'est le point le plus flagrant de votre terminal.
- **Explication :** * Temps avec jointure ($lookup) : **1867 ms**
    - Temps sans jointure : **11 ms**

- **Analyse :** La jointure multiplie le temps d'exécution par environ **170**. Cela démontre que si $lookup est puissant pour croiser les données (ex: lier les ventes aux secteurs industriels des entreprises), il est extrêmement coûteux en ressources. C'est la preuve mathématique que pour du Big Data massif, il vaut mieux privilégier des données "dénormalisées" (déjà regroupées) ou utiliser des index sur les champs de jointure.

5. Interprétation des résultats :

Mission A : Top 10 Produits (Ventes & Notes)

- **Volume de Ventes :** Le produit leader est le *"Personalized Cat Dog ID Tag"* avec plus de **15 millions d'unités**. C'est un outlier massif qui indique soit un produit viral, soit une donnée de test/agrégée.
- **Tendance de consommation :** On remarque que les accessoires de personnalisation (médailles gravées) et de confort (gamelles, lits) dominent le marché.
- **Qualité (Rating) :** Plusieurs produits affichent une note parfaite de **5.0**. Fait intéressant : les produits les mieux notés ne sont pas nécessairement ceux les plus vendus, ce qui montre que la satisfaction client est élevée même sur des produits de niche (ex: accessoires de mariage pour chiens).

Mission B : Statistiques Globales (Nettoyage)

- **Panier Moyen :** Environ **144.76 USD**. C'est une valeur élevée qui suggère un dataset orienté vers des produits premium ou des achats groupés.
- **Démographie :** L'âge moyen des acheteurs est de **44 ans**, ce qui indique une clientèle adulte avec un pouvoir d'achat stable.
- **Santé des données :** Le pipeline a traité **4494 documents** valides. Le revenu total s'élève à **650 574.97 USD**.

Mission C : Analyse Croisée (Industry x Category)

- **Secteur Dominant :** Le secteur **"Financial Data & Stock Exchanges"** génère le plus gros chiffre d'affaires, particulièrement dans la catégorie **"Clothing"** (230 854 USD).
- **Corrélation intéressante :** On observe que les secteurs à hauts revenus (Finance, Capital Markets, Oil & Gas) ont des paniers moyens par commande (avgOrderValue) très élevés, dépassant parfois les **800 USD** dans l'électronique.
- **Performance :** L'analyse avec jointure ($lookup) est beaucoup plus granulaire mais produit des résultats plus complexes à agréger que l'analyse simple.

5. Remarques et Observations Critiques :

Mission	Opération	Temps d'exécution
Mission A	Top 10 par Quantité	37 ms
Mission A	Top 10 par Note	15 ms
Mission B	Calcul statistiques	68 ms
Mission B	Nettoyage + Sauvegarde ($out)	230 ms
Mission C	Analyse croisée (avec $lookup)	1867 ms
Mission C	Analyse simple (sans jointure)	11 ms

1. **Impact de la Jointure ($lookup) :** L'analyse sans jointure montre 5 catégories simples, tandis que la version avec jointure multiplie les segments (Secteur x Catégorie). **Observation :** Bien que plus riche, la jointure augmente le volume de données traitées en mémoire (on passe de 5 lignes à 20+ lignes de résultats).
2. **Qualité des Données (Dirty Data) :** Le fait que le count dans la mission A soit parfois supérieur à 1 (ex: count: 3 pour les bols en inox) prouve que le dataset contient des doublons de titres. Le $group a été essentiel pour consolider ces lignes éparpillées en une seule valeur statistique réelle.
3. **Hétérogénéité des types :** Le script Python a dû forcer des conversions (to_int, to_float). Sans cette étape, les fonctions $avg et $sum de MongoDB auraient renvoyé 0 ou null, car le moteur ne peut pas additionner des chaînes de caractères.
4. **Observation sur le "Top 10" :** Certains produits ont des noms extrêmement longs. Dans une application réelle, il serait judicieux d'ajouter une étape $project pour tronquer les titres ou utiliser un identifiant unique (SKU) pour la lisibilité.
5. **Coût de l'écriture disque :** Le passage de 68 ms (calcul simple) à 230 ms pour la Mission B montre l'impact de l'étape $out. L'écriture d'une nouvelle collection sur le disque prend plus de temps que la simple lecture en mémoire.
6. **Gestion de la mémoire :** Bien que vous n'ayez pas eu de message d'erreur "Sort exceeded memory limit", la présence de l'option allowDiskUse: true dans vos scripts est ce qui a garanti le succès de l'opération, notamment pour la Mission C qui manipule deux collections simultanément.

7. Conclusion :

Ce TP nous a permis de démontrer la puissance de **MongoDB** comme outil analytique pour le Big Data, allant au-delà du simple stockage de documents.

- **Maîtrise technique :** Nous avons validé l'utilisation du **Framework d'Agrégation** pour transformer des données brutes et "sales" en informations décisionnelles.
- **Flexibilité :** Le passage de Python (pour l'import/nettoyage primaire) à JavaScript/NoSQL (pour l'analyse complexe) montre la complémentarité des outils dans un pipeline de données moderne.
- **Recommandation :** Pour des volumes de données encore plus massifs, l'indexation des champs de jointure (location et city) serait l'étape suivante indispensable pour maintenir ces performances de traitement.